Document number: LEWG, SG14, SG6: P0037R3
Date: 2016-10-17
Reply-to: John McFarlane, mcfarlane.john+fixed-point@gmail.com
Reply-to: Michael Wong, fraggamuffin@gmail.com
Audience: SG6, SG14, LEWG

Fixed-Point Real Numbers

I. Introduction

This proposal introduces a system for performing binary fixed-point arithmetic using integral types.

II. Motivation

Floating-point types are an exceedingly versatile and widely supported method of expressing real numbers on modern architectures.

However, there are certain situations where fixed-point arithmetic is preferable:

- Some systems lack native floating-point registers and must emulate them in software;
- many others are capable of performing some or all operations more efficiently using integer arithmetic;
- certain applications can suffer from the variability in precision which comes from a dynamic radix point [1];
- in situations where a variable exponent is not desired, it takes valuable space away from the significand and reduces precision and
- not all hardware and compilers produce exactly the same results, leading to non-deterministic results.

Integer types provide the basis for an efficient representation of binary fixed-point real numbers. However, laborious, error-prone steps are required to normalize the results of certain operations and to convert to and from fixed-point types.

A set of tools for defining and manipulating fixed-point types is proposed. These tools are designed to make work easier for those who traditionally use integers to perform low-level, high-performance fixed-point computation. They are composable such that a wide range of trade-offs between speed, accuracy and safety are supported.

III. Impact On the Standard

This proposal is a pure library extension. It does not require changes to any standard classes or functions. It adds several new class and function templates to new header file, `<fixed_point>`.

It depends on two new class templates, `width<class>` and `set_width<class, int>`, added to existing header file, `<cstdint>` and proposed in P0381 [10].

IV. Design Decisions

The design is driven by the following aims in roughly descending order:

1. to automate the task of using integer types to perform low-level binary fixed-point arithmetic;
2. to facilitate a style of code that is intuitive to anyone who is comfortable with integer and floating-point arithmetic;
3. to treat fixed-point as a super-set of integer such that a fixed-point type with an exponent of zero can provide a drop-in replacement for its underlying integer type
4. to avoid incurring expense for unused features - including compilation time.

More generally, the aim of this proposal is to contain within a single API all the tools necessary to perform binary fixed-point arithmetic. The design facilitates a wide range of competing compile-time strategies for avoiding overflow and precision loss, but implements only the simplest by default. Similarly, orthogonal concerns such as run-time overflow detection and rounding modes are deferred to the underlying integer types used as storage.

Class Template

Fixed-point numbers are specializations of

```
template <class Rep, int Exponent>
class fixed_point;
```

where the template parameters are described as follows.

`Rep` Type Template Parameter

This parameter identifies the capacity and signedness of the underlying type used to represent the value. In other words, the size of the resulting type will be `sizeof(Rep)` and it will be signed iff `std::numeric_limits<Rep>::is_signed` is true.

`Rep` may be a fundamental integral type or similar integer-like type. The most suitable types are: `std::int8_t`, `std::uint8_t`, `std::int16_t`, `std::uint16_t`, `std::int32_t` and `std::uint32_t`. In limited situations, `std::int64_t` and `std::uint64_t` can be used. The reasons for these limitations relate to the difficulty in finding a type that is suitable for performing lossless integer division.

The characteristics of `Rep` are passed to the fixed-point type. If, for example, `Rep` has an alternative rounding style, overflow handling strategy or large storage capacity, then the `fixed_point` specialization will benefit from this feature. By defaulting to `int` for its representation, `fixed_point` defaults to machine-level efficiency and minimal compile-time overhead.

`Exponent` Non-Type Template Parameter

The exponent of a fixed-point type is the equivalent of the exponent field in a floating-point type and shifts the stored value by the requisite number of bits necessary to produce the desired range. The default value of `Exponent` is zero, giving `fixed_point<T>` the same range as `T`. By far the most common use of fixed-point is to store values with fractional digits. Thus, the exponent is typically a negative value.

The resolution of a specialization of `fixed_point` is

```
pow(2, Exponent)
```

and the minimum and maximum values are

```
std::numeric_limits<Rep>::min() * pow(2, Exponent)
```

and

```
std::numeric_limits<Rep>::max() * pow(2, Exponent)
```

respectively.

Any usage that results in values of `Exponent` which lie outside the range, (`INT_MIN / 2`, `INT_MAX / 2`), may result in undefined behavior and/or overflow or underflow. This range of exponent values is far in excess of the largest built-in floating-point type and should be adequate for all intents and purposes.

`make_fixed` and `make_ufixed` Helper Types

The `Exponent` template parameter is versatile and concise. It is an intuitive scale to use when considering the full range of positive and negative exponents a fixed-point type might possess. It also corresponds to the exponent field of built-in floating-point types.

However, most fixed-point formats can be described more intuitively by the cardinal number of integer and/or fractional digits they contain. Most users will prefer to distinguish fixed-point types using these parameters.

For this reason, two aliases are defined in the style of `make_signed`.

These aliases are declared as:

```
template <int IntegerDigits, int FractionalDigits = 0, class Archetype = signed>
using make_fixed;
```

and

```
template <int IntegerDigits, int FractionalDigits = 0, class Archetype = unsigned>
using make_ufixed;
```

They resolve to a `fixed_point` specialization with the given signedness and number of integer and fractional digits. They may contain additional integer digits.

For example, one could define and initialize an 8-bit, unsigned, fixed-point variable with four integer digits and four fractional digits:

```
make_ufixed<4, 4> value { 15.9375 };
```

or a 32-bit, signed, fixed-point number with two integer digits and 29 fractional digits:

```
make_fixed<2, 29> value { 3.141592653 };
```

Type parameter, `Archetype`, is provided in the case that a `fixed_point` specialization is desired which has as the `Rep` parameter some type other than a built-in integral. The signedness of `Archetype` corresponds to the signedness of the resultant `fixed_point`

specialization although the size does not.

Conversion

Fixed-point numbers can be converted to and from other arithmetic types. A precondition is that the conversion is supported by the `Rep` type. Implicit conversion is allowed in cases where the destination type can store all values of the source type.

For example, this implicit initialization is valid because all values of `uint8_t` can be stored in `a`

```
fixed_point<uint16_t, -8> a = uint8_t{};
```

but none of these implicit initializations compile:

```
fixed_point<uint16_t, -8> b = uint16_t{};  // maximum value exceeded
fixed_point<uint16_t, -8> c = int8_t{};  // minimum value exceeded
fixed_point<uint16_t, -8> d = fixed_point<uint16_t, -16>{};  // precision loss
uint16_t e = fixed_point<uint16_t, -8>{};  // precision loss
fixed_point<uint16_t, -8> f = double{};  // all of the above
```

While effort is made to ensure that significant digits are not lost during conversion, no effort is made to avoid rounding errors. Whatever would happen when converting to and from `Rep` largely applies to `fixed_point` objects also. For example:

```
fixed_point<int, -1>{.499}==0.0
```

...equates to `true` and is considered an acceptable rounding error.

Operator Overloads

Any operators that might be applied to integer types can also be applied to fixed-point types. A guiding principle of operator overloads is that they perform as little run-time computation as is practically possible.

With the exception of shift and comparison operators, binary operators can take any combination of:

- one or two fixed-point arguments and
- zero or one arguments of any arithmetic type, i.e. a type for which `numeric_limits` is specialized.

Where the inputs are not identical fixed-point types, a simple set of promotion-like rules are applied to determine the return type:

1. If one of the arguments is a floating-point type, then the type of the result is the smallest floating-point type of equal or greater size than the inputs.
2. If one of the arguments is an integral type, then it is converted to a `fixed_point<>` type with `Exponent` zero such that rule 3) applies ...
3. If both arguments are fixed-point then:
    1. If the operation is addition or subtraction then the operand with the greater exponent is converted to a type with the same exponent as the other operand and then 3) is applied.
    2. If the operation is division then the left-hand operand is widened and padded with as many low-order bits as the right-hand operand has integer bits and then 3) is applied.
    3. Then for all operations, the arithmetic operation is performed on the underlying integers and returned as a fixed-point type with the correct Exponent and without any extra shift operations being performed on the result.

Some examples:

```
fixed_point<uint8_t, -3>{8} + fixed_point<int8_t, -4>{3} === fixed_point<int, -4>{11}
fixed_point<uint8_t, -3>{8} + 3 === fixed_point<int, -3>{11}
fixed_point<uint8_t, -3>{8} + float{3} === float{11}
```

The reasoning behind this choice is a combination of predictability and performance. It is explained for each rule as follows:

1. ensures that the least computation is performed where fixed-point types are used exclusively. Aside from division requiring shift operations, should require similar computational costs to equivalent integer operations;
2. loosely follows the promotion rules for mixed-mode arithmetic, ensures values with exponents far beyond the range of the fixed-point type are catered for and avoids costly conversion from floating-point to integer.

A guiding aim is for specializations with `Exponent` set to 0 to behave as closely as possible like their `Rep`. For instance, where possible, an object of type, `int`, should be interchangeable with `fixed_point<>`.

Shift operator overloads require an integer type as the right-hand parameter.

Comparison operators convert the inputs to a common result type following rules similar to addition and subtraction above before performing a comparison and returning `true` or `false`.

Overflow

Because arithmetic operators return a result of equal capacity to their inputs, they carry a risk of overflow. For instance,

```
make_ufixed<2, 30>(3) + make_ufixed<2, 30>(1)
```

is zero on architectures where `int` is 4 bytes because a type with 2 integer bits cannot store a value of 4.

The result of overflow of any bits in a fixed-point value depends entirely on how `Rep` handles overflow. Thus, for built-in signed types, the result is undefined and for built-in unsigned types, the value wraps around.

Underflow

The other typical cause of lost bits is underflow where, for example,

```
15 / 2
```

results in a value of 7.

Unlike the other arithmetic operators which aim to get as close to machine perfomance as possible, the division operator widens and pads its operands such that:

```
make_fixed<7, 0>(15) / make_fixed<7, 0>(2)
```

results in a type of `fixed_point<int, -7>` and a value of 7.5. However, division can still result in loss of precision due to the limitations of representing rational numbers using digital systems.

Dealing With Overflow and Flushes

Errors resulting from overflow and flushes are two of the biggest headaches related to fixed-point arithmetic. Integers suffer the same kinds of errors but are somewhat easier to reason about as they lack fractional digits. Floating-point numbers are largely shielded from these errors by their variable exponent and implicit bit.

Four strategies for avoiding overflow in fixed-point types are presented:

1. simply leave it to the user to avoid overflow;
2. allow the user to provide a custom type for `Rep` which behaves differently from built-in integral types;
3. promote the result to a larger type to ensure sufficient capacity or
4. adjust the exponent of the result upward to ensure that the top limit of the type is sufficient to preserve the most significant digits at the expense of the less significant digits.

For most arithmetic operators, a combination of choices 1) and 3) is taken because it most closely follows the behavior of integer types. Thus it should cause the least surprise to the fewest users. This makes it far easier to reason about in code where functions are written with a particular type in mind. It also requires the least computation in most cases.

Choice 2) is beyond the scope of this proposal and is covered in more detail in the section, **Alternative Types for `Rep`**.

Choice 4) is reasonably robust to overflow events. However, it represent different trade-offs and is less portable as it is sensitive to the width of the `Rep` type.

Named Arithmetic Functions

The following named function templates can be used as alternatives to arithmetic operators, `-`, `+`, `*` and `/`.

```
template<class Rep, int Exponent>
constexpr auto negate(const fixed_point<RhsRep, RhsExponent>& rhs);

template<class Lhs, class Rhs>
constexpr auto add(const Lhs& lhs, const Rhs& rhs);

template<class Lhs, class Rhs>
constexpr auto subtract(const Lhs& lhs, const Rhs& rhs);

template<class Lhs, class Rhs>
constexpr auto multiply(const Lhs& lhs, const Rhs& rhs);

template<class Lhs, class Rhs>
constexpr auto divide(const Lhs& lhs, const Rhs& rhs);
```

They eschew machine-friendly integer promotion in favor of custom widening rules that place conciseness and correctness before raw performance.

Multiplying two 8-bit values does not necessarily produce an int-sized result:

```
auto f = fixed_point<uint8_t, -4>{15.9375};
auto p = multiply(f, f);
// p === fixed_point<uint16_t, -8>{254.00390625}
```

The arithmetic operators shield the user from none of the pitfalls of fixed-point arithmetic. For example, naive use of `*` can easily lead to surprising results.

```
auto f = fixed_point<unsigned, -28>{15.9375};
auto p = f * f;
// p === fixed_point<unsigned, -56>{0}
```

In contrast, the `multiply` function returns appropriately widened results. It does a better job at avoiding unnecessary narrowing and catastrophic information loss.

```
auto f = fixed_point<unsigned, -28>{15.9375};
auto p = multiply(f, f);
// p === fixed_point<uint64_t, -56>{254.00390625}
```

Functions, `add` and `subtract` go to similar lengths to preserve results:

```
auto a1 = fixed_point<int8_t, 32>{0x7f00000000LL};
auto a2 = fixed_point<int8_t, 0>{0x7f};
auto s = add(a1, a2);
// s === fixed_point<int64_t, 0>{0x7f0000007fLL}
```

Function, `divide`, however serves a different purpose. Operator `/` behaves very differently than `*` and does not provide access to machine-efficient integer division. So `divide` is given that job instead. The reason why the user is protected from integer division quickly becomes apparent:

```
auto n = fixed_point<uint32_t, -16>{1};
auto d = fixed_point<uint32_t, -16>{2};

auto q1 = n / d;
// q1 === fixed_point<uint64_t, -32>{0.5}

auto q2 = divide(n, d);
// q2 === fixed_point<uint32_t, 0>{0}
```

Alternative Types for `Rep`

Using built-in integral types as the default underlying representation minimizes certain costs:

- many fixed-point operations are as efficient as their integral equivalents;
- compile-time complexity is kept relatively low and
- the behavior of fixed-point types should cause few surprises.

However, this choice also brings with it many of the deficiencies of built-in types. For example:

- the typical rounding behavior is distinct for:
  - conversion from floating-point types;
  - right shift and
  - divide operations;
- all of these rounding behaviors cause drift and propagate error;
- overflow, underflow and flush are handled silently with wrap-around or undefined behavior;
- divide-by-zero similarly results in undefined behavior and
- the range of values is limited by the largest type: `long long int`.

The effort involved in addressing these deficiencies is non-trivial and on-going (for example [2]). As solutions are made available, it should become easier to define custom integral types which address concerns surrounding robustness and correctness. Such types deserve their place in the standard library.

Example Custom Type, `integer`

A composable system of integer types that is suitable for use with `fixed_point` might take the following form:

```
// size may be rounded up in some cases;
template <int NumBytes, bool IsSigned = true>
class sized_integer;

// may take built-in or sized_integer as Rep parameter
template <class Rep = int, rounding Rounding = rounding::towards_odd>
```

```
class rounding_integer;

// may take built-in, sized_integer or rounding_integer as Rep parameter
template <class Rep = int, overflow Overflow = overflow::exception>
class overflow_integer;

// a 'kitchen sink' custom integer type
template <int NumBytes, bool IsSigned = true, rounding Rounding = rounding::towards_odd, overflow Overflow = overflow::exception>
using integer =
  overflow_integer<
    rounding_integer<
      sized_integer<NumBytes, IsSigned>,
      Rounding>,
    Overflow>;
```

Any of these types might be used to compose `fixed_point` specializations without paying (in compile-time complexity) for features that are not used.

While the issues related to integer types affect the fixed-point types they support, they are not specific to fixed-point. It would not only be premature - but inappropriate - to attempt to address rounding and error handling at the level of a fixed-point type.

Required Specializations

For a type to be suitable as parameter, `Rep`, of `fixed_point`, it must meet the following requirements:

- it must have specialized the following existing standard library types:
  - `numeric_limits`
  - `make_signed` and `make_unsigned`
- it must have specialized the following proposed standard library types:
  - `width` and `set_width` as described in P0381 [10]

Note that `make_signed` and `make_unsigned` must not be specialized for custom types. Unless this rule can be relaxed, some equivalent mechanism must be introduced in order for custom types to be used with `fixed_point<>`. One possibility is the addition of `numeric_limits<>::signed` and `numeric_limits<>::unsigned` type aliases.

Example

The following function, `magnitude`, calculates the magnitude of a 3-dimensional vector.

```
template<class Fp>
constexpr auto magnitude(Fp x, Fp y, Fp z)
{
    return sqrt(x*x+y*y+z*z);
}
```

And here is a call to `magnitude`.

```
auto m = magnitude(
        fixed_point<uint16_t, -12>(1),
        fixed_point<uint16_t, -12>(4),
        fixed_point<uint16_t, -12>(9));
// m === fixed_point<uint32_t, -24>{9.8994948863983154}
```

V. Technical Specification

Header <fixed_point> Synopsis

```
namespace std {
  template <class Rep, int Exponent> class fixed_point;

  template <int IntegerDigits, int FractionalDigits = 0, class Archetype = signed>
    using make_fixed;
  template <int IntegerDigits, int FractionalDigits = 0, class Archetype = unsigned>
    using make_ufixed;

  template <class Rep, int Exponent>
    constexpr bool operator==(
      const fixed_point<Rep, Exponent> & lhs,
      const fixed_point<Rep, Exponent> & rhs) noexcept;
  template <class Rep, int Exponent>
    constexpr bool operator!=(
      const fixed_point<Rep, Exponent> & lhs,
      const fixed_point<Rep, Exponent> & rhs) noexcept;
  template <class Rep, int Exponent>
```

```
  constexpr bool operator<(
    const fixed_point<Rep, Exponent> & lhs,
    const fixed_point<Rep, Exponent> & rhs) noexcept;
template <class Rep, int Exponent>
  constexpr bool operator>(
    const fixed_point<Rep, Exponent> & lhs,
    const fixed_point<Rep, Exponent> & rhs) noexcept;
template <class Rep, int Exponent>
  constexpr bool operator>=(
    const fixed_point<Rep, Exponent> & lhs,
    const fixed_point<Rep, Exponent> & rhs) noexcept;
template <class Rep, int Exponent>
  constexpr bool operator<=(
    const fixed_point<Rep, Exponent> & lhs,
    const fixed_point<Rep, Exponent> & rhs) noexcept;

template <class Rep, int Exponent>
  constexpr fixed_point<Rep, Exponent> operator-(
    const fixed_point<Rep, Exponent> & rhs) noexcept;
template <class Rep, int Exponent>
  constexpr fixed_point<Rep, Exponent> operator+(
    const fixed_point<Rep, Exponent> & lhs,
    const fixed_point<Rep, Exponent> & rhs) noexcept;
template <class Rep, int Exponent>
  constexpr fixed_point<Rep, Exponent> operator-(
    const fixed_point<Rep, Exponent> & lhs,
    const fixed_point<Rep, Exponent> & rhs) noexcept;
template <class Rep, int Exponent>
  fixed_point<Rep, Exponent> & operator+=(
    fixed_point<Rep, Exponent> & lhs,
    const fixed_point<Rep, Exponent> & rhs) noexcept;
template <class Rep, int Exponent>
  fixed_point<Rep, Exponent> & operator-=(
    fixed_point<Rep, Exponent> & lhs,
    const fixed_point<Rep, Exponent> & rhs) noexcept;
template <class Rep, int Exponent>
  fixed_point<Rep, Exponent> & operator*=(
    fixed_point<Rep, Exponent> & lhs,
    const fixed_point<Rep, Exponent> & rhs) noexcept;
template <class Rep, int Exponent>
  fixed_point<Rep, Exponent> & operator/=(
    fixed_point<Rep, Exponent> & lhs,
    const fixed_point<Rep, Exponent> & rhs) noexcept;

template <class Lhs, class Rhs>
  constexpr auto operator==(const Lhs & lhs, const Rhs & rhs) noexcept;
template <class Lhs, class Rhs>
  constexpr auto operator!=(const Lhs & lhs, const Rhs & rhs) noexcept;
template <class Lhs, class Rhs>
  constexpr auto operator<(const Lhs & lhs, const Rhs & rhs) noexcept;
template <class Lhs, class Rhs>
  constexpr auto operator>(const Lhs & lhs, const Rhs & rhs) noexcept;
template <class Lhs, class Rhs>
  constexpr auto operator>=(const Lhs & lhs, const Rhs & rhs) noexcept;
template <class Lhs, class Rhs>
  constexpr auto operator<=(const Lhs & lhs, const Rhs & rhs) noexcept;

template <class Lhs, class Rhs>
  constexpr auto operator+(
    const Lhs & lhs,
    const Rhs & rhs) noexcept;
template <class Lhs, class Rhs>
  constexpr auto operator-(
    const Lhs & lhs,
    const Rhs & rhs) noexcept;
template <class LhsRep, int LhsExponent, class RhsRep, int RhsExponent>
  constexpr auto operator*(
    const fixed_point<LhsRep, LhsExponent> & lhs,
    const fixed_point<RhsRep, RhsExponent> & rhs) noexcept;
template <class LhsRep, int LhsExponent, class RhsRep, int RhsExponent>
  constexpr auto operator/(
    const fixed_point<LhsRep, LhsExponent> & lhs,
    const fixed_point<RhsRep, RhsExponent> & rhs) noexcept;
template <class LhsRep, int LhsExponent, class Integer>
  constexpr auto operator*(
    const fixed_point<LhsRep, LhsExponent> & lhs,
    const Integer & rhs) noexcept;
template <class LhsRep, int LhsExponent, class Integer>
  constexpr auto operator/(
    const fixed_point<LhsRep, LhsExponent> & lhs,
```

```
    const Integer & rhs) noexcept;
  template <class Integer, class RhsRep, int RhsExponent>
    constexpr auto operator*(
      const Integer & lhs,
      const fixed_point<RhsRep, RhsExponent> & rhs) noexcept;
  template <class Integer, class RhsRep, int RhsExponent>
    constexpr auto operator/(
      const Integer & lhs,
      const fixed_point<RhsRep, RhsExponent> & rhs) noexcept;
  template <class LhsRep, int LhsExponent, class Float>
    constexpr auto operator*(
      const fixed_point<LhsRep, LhsExponent> & lhs,
      const Float & rhs) noexcept;
  template <class LhsRep, int LhsExponent, class Float>
    constexpr auto operator/(
      const fixed_point<LhsRep, LhsExponent> & lhs,
      const Float & rhs) noexcept;
  template <class Float, class RhsRep, int RhsExponent>
    constexpr auto operator*(
      const Float & lhs,
      const fixed_point<RhsRep, RhsExponent> & rhs) noexcept;
  template <class Float, class RhsRep, int RhsExponent>
    constexpr auto operator/(
      const Float & lhs,
      const fixed_point<RhsRep, RhsExponent> & rhs) noexcept;
  template <class LhsRep, int Exponent, class Rhs>
    fixed_point<LhsRep, Exponent> & operator+=(fixed_point<LhsRep, Exponent> & lhs, const Rhs & rhs) noexcept;
  template <class LhsRep, int Exponent, class Rhs>
    fixed_point<LhsRep, Exponent> & operator-=(fixed_point<LhsRep, Exponent> & lhs, const Rhs & rhs) noexcept;
  template <class LhsRep, int Exponent>
  template <class Rhs, typename std::enable_if<std::is_arithmetic<Rhs>::value, int>::type Dummy>
    fixed_point<LhsRep, Exponent> &
    fixed_point<LhsRep, Exponent>::operator*=(const Rhs & rhs) noexcept;
  template <class LhsRep, int Exponent>
  template <class Rhs, typename std::enable_if<std::is_arithmetic<Rhs>::value, int>::type Dummy>
    fixed_point<LhsRep, Exponent> &
    fixed_point<LhsRep, Exponent>::operator/=(const Rhs & rhs) noexcept;
  template <class Rep, int Exponent>
    constexpr fixed_point<Rep, Exponent>
      sqrt(const fixed_point<Rep, Exponent> & x) noexcept;
}
```

`fixed_point<>` Class Template

```
template <class Rep = int, int Exponent = 0>
class fixed_point
{
public:
  using rep = Rep;

  constexpr static int exponent;
  constexpr static int digits;
  constexpr static int integer_digits;
  constexpr static int fractional_digits;

  fixed_point() noexcept;
  template <class S, typename std::enable_if<_impl::is_integral<S>::value, int>::type Dummy = 0>
    explicit constexpr fixed_point(S s) noexcept;
  template <class S, typename std::enable_if<std::is_floating_point<S>::value, int>::type Dummy = 0>
    explicit constexpr fixed_point(S s) noexcept;
  template <class FromRep, int FromExponent>
    explicit constexpr fixed_point(const fixed_point<FromRep, FromExponent> & rhs) noexcept;
  template <class S, typename std::enable_if<_impl::is_integral<S>::value, int>::type Dummy = 0>
    fixed_point & operator=(S s) noexcept;
  template <class S, typename std::enable_if<std::is_floating_point<S>::value, int>::type Dummy = 0>
    fixed_point & operator=(S s) noexcept;
  template <class FromRep, int FromExponent>
    fixed_point & operator=(const fixed_point<FromRep, FromExponent> & rhs) noexcept;

  template <class S, typename std::enable_if<_impl::is_integral<S>::value, int>::type Dummy = 0>
    explicit constexpr operator S() const noexcept;
  template <class S, typename std::enable_if<std::is_floating_point<S>::value, int>::type Dummy = 0>
    explicit constexpr operator S() const noexcept;
  explicit constexpr operator bool() const noexcept;

  template <class Rhs, typename std::enable_if<std::is_arithmetic<Rhs>::value, int>::type Dummy = 0>
    fixed_point &operator*=(const Rhs & rhs) noexcept;
  template <class Rhs, typename std::enable_if<std::is_arithmetic<Rhs>::value, int>::type Dummy = 0>
    fixed_point & operator/=(const Rhs & rhs) noexcept;
```

```
  constexpr rep data() const noexcept;
  static constexpr fixed_point from_data(rep r) noexcept;
};
```

VI. Future Issues

Library Support

Because the aim is to provide an alternative to existing arithmetic types which are supported by the standard library, it is conceivable that a future proposal might specialize existing class templates and overload existing functions.

Possible candidates for overloading include the functions defined in <cmath> and a templated specialization of `numeric_limits`. A new type trait, `is_fixed_point`, would also be useful.

While `fixed_point` is intended to provide drop-in replacements to existing built-ins, it may be preferable to deviate slightly from the behavior of certain standard functions. For example, overloads of functions from <cmath> will be considerably less concise, efficient and versatile if they obey rules surrounding error cases. In particular, the guarantee of setting `errno` in the case of an error prevents a function from being defined as pure. This highlights a wider issue surrounding the adoption of the functional approach and compile-time computation that is beyond the scope of this document.

One suggested addition is a specialization of `std::complex`. This would take the form:

```
template<class Rep, int Exponent>
class complex<fixed_point<Rep, Exponent>>;
```

This type's arithmetic operators would differ from existing specializations because `fixed_point<>` operators often return results of a different type to their operands. Hence signatures such as

```
template<class T>
complex<T> operator*( const complex<T>& lhs, const complex<T>& rhs);
```

would need to be replaced with:

```
template<class T>
auto operator*( const complex<T>& lhs, const complex<T>& rhs);
```

Compile-Time Bit-Shift Operations

A notable feature of the *fp* library [4] is the creation of an alias for `integral_constant` which can be applied to the right-hand side of bit-shift operations. The type returned from this operation has identical bit-wise value to the left-hand input but with `Exponent` value adjusted by the amount of the right-hand side. It is essentially the same as the `trunc_shift_` functions and means that when shifting by literal values what looks looks like run-time operation is a compile-time calculation which guarantees no overflow or underflow.

Alternative Return Type Policies

When devising a strategy for mitigating the risk of overflow during arithmetic operations, the number of integer and fractional bits stored in the result is an important choice. The `fixed_point` type picks one of the simpler options by default, but it is by no means the only viable one.

The *fp* library [4] returns a type whose size matches the inputs but whose exponent is shifted to preserve high bits. The arithmetic types proposed in P0106 [8] increase capacity to ensure that precision is preserved. Even greater control of the required capacity of a fixed-point type can be afforded by systems such as the *bounded::integer* library [3].

A common requirement among these approaches is the ability to specify the return type of arithmetic operations. For this reason, named-function arithmetic operators which are more expressive than those proposed so far may be necessary.

These functions would specify a return type as a template parameter. For example:

```
template <class Result, class Lhs, class Rhs>
constexpr Result fixed_point_multiply(const Lhs & lhs, const Rhs & rhs);
```

Non-Binary Radixes

Interest in decimal fixed-point arithmetic has been observed. It seems plausible that a general-purpose type could support both binary and decimal fixed-point types as follows:

```
template<class Rep, int Exponent, int Radix>
class basic_fixed_point;
```

```
template<class Rep, int Exponent>
using fixed_point = basic_fixed_point<Rep, Exponent, 2>;

template<class Rep, int Exponent>
using decimal_fixed_point = basic_fixed_point<Rep, Exponent, 10>;
```

This naming scheme imitates `basic_string` in order to illustrate a similar pattern.

Further investigation needs to be conducted in order to ascertain whether this break-down can maintain the same level of efficiency.

VII. Prior Art

Many examples of fixed-point support in C and C++ exist. While almost all of them aim for low run-time cost and expressive alternatives to raw integer manipulation, they vary greatly in detail and in terms of their interface.

One especially interesting dichotomy is between solutions which offer a discrete selection of fixed-point types and libraries which contain a continuous range of exponents through type parameterization.

N1169

One example of the former is found in proposal N1169 [5], the intent of which is to expose features found in certain embedded hardware. It introduces a succinct set of language-level fixed-point types and impose constraints on the number of integer or fractional digits each can possess.

As with all examples of discrete-type fixed-point support, the limited choice of exponents is a considerable restriction on the versatility and expressiveness of the API.

Nevertheless, it may be possible to harness performance gains provided by N1169 fixed-point types through explicit template specialization. This is likely to be a valuable proposition to potential users of the library who find themselves targeting platforms which support fixed-point arithmetic at the hardware level.

P0106

There are many other C++ libraries available which fall into the latter category of continuous-range fixed-point arithmetic [4] [6] [7]. In particular, an existing library proposal by Lawrence Crowl, P0106 [8] (formerly N3352), aims to achieve very similar goals through similar means and warrants closer comparison than N1169.

P0106 introduces four class templates covering the quadrant of signed versus unsigned and fractional versus integer numeric types. It is intended to replace built-in types in a wide variety of situations and accordingly, is highly compile-time configurable in terms of how rounding and overflow are handled. Parameters to these four class templates include the storage in bits and - for fractional types - the resolution.

The `fixed_point` class template could probably - with a few caveats - be generated using the two fractional types, `nonnegative` and `negatable`, replacing the `Rep` parameter with the integer bit count of `Rep`, specifying `fastest` for the rounding mode and specifying `undefined` as the overflow mode.

However, `fixed_point` more closely and concisely caters to the needs of users who already use integer types and simply desire a more concise, less error-prone form. It more closely follows the four design aims of the library and - it can be argued - more closely follows the spirit of the standard in its pursuit of zero-cost abstraction.

Some aspects of the design of the P0106 API which back up these conclusion are that:

- the result of arithmetic operations closely resemble the `trunc_` function templates and are potentially more costly at run-time;
- the nature of the range-specifying template parameters - through careful framing in mathematical terms - abstracts away valuable information regarding machine-critical type size information;
- the breaking up of duties amongst four separate class templates introduces four new concepts and incurs additional mental load for relatively little gain while further detaching the interface from vital machine-level details and
- the absence of the most negative number from signed types reduces the capacity of all types by one.

The added versatility that the P0106 API provides regarding rounding and overflow handling are of relatively low priority to users who already bear the scars of battles with raw integer types. Nevertheless, providing them as options to be turned on or off at compile time is an ideal way to leave the choice in the hands of the user.

Many high-performance applications - in which fixed-point is of potential value - favor run-time checks during development which are subsequently deactivated in production builds. The P0106 interface is highly conducive to this style of development. The design proposed in this paper aims to achieve similar results by composing fixed-point types from custom integral types.

VIII. Acknowledgements

SG6: Lawrence Crowl SG14: Guy Davidson, Michael Wong
Contributors: Ed Ainsley, Billy Baker, Lance Dyson, Marco Foco, Mathias Gaunard, ClÃ©ment GrÃ©goire, Nicolas Guillemot, Kurt Guntheroth, Matt Kinzelman, JoÃ«l Lamotte, Sean Middleditch, Paul Robinson, Patrice Roy, Peter Schregle, Ryhor Spivak

IX. Revisions

This paper revises P0037R2:

- `width`, `set_width` etc. are moved from <type_traits> to <cstdint>;
- greater use of `numeric_limits` in place of definitions from <type_traits>;
- mention that exponents are typicaly negative;
- stricted rules regarding implicit conversion to and from integer types;
- revised rules for mixed-mode operator overloads;
- change in behavior of arithmetic operators and named arithmetic functions;
- brief discussion of `std::complex` specialization;
- clearer reference to P0106R0, author and previous paper number.

X. References

1. Why Integer Coordinates?, http://www.pathengine.com/Contents/Overview/FundamentalConcepts/WhyIntegerCoordinates/page.php
2. Rounding and Overflow in C++, http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2015/p0105r0.html
3. C++ bounded::integer library, http://doublewise.net/c++/bounded/
4. fp, C++14 Fixed Point Library, https://github.com/mizvekov/fp
5. N1169, Extensions to support embedded processors, http://www.open-std.org/JTC1/SC22/WG14/www/docs/n1169.pdf
6. fpmath, Fixed Point Math Library, http://www.codeproject.com/Articles/37636/Fixed-Point-Class
7. Boost fixed_point (proposed), Fixed point integral and fractional types, https://github.com/viboes/fixed_point
8. P0106, C++ Binary Fixed-Point Arithmetic, http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2015/p0106r0.html
9. fixed_point, Reference Implementation of P0037, https://github.com/johnmcfarlane/fixed_point
10. P0381, Numeric Width, http://johnmcfarlane.github.io/fixed_point/papers/p0381r1.html

XI. Appendix 1: Reference Implementation

An in-development implementation of the fixed_point class template and its essential supporting functions and types is available [9].

Items include:

- utility header containing definitions for:
  - math and trigonometric functions and
  - a partial `numeric_limits` specialization;
- a type, `integer`, intended to explore and illustrate the potential of custom `Rep`;
- a type, `elastic`, intended to illustrate the use of `fixed_point` to design better-behaved numeric types such as those presented in P0106 [8];
- compile-time tests of `constexpr` operations;
- run-time tests of assignment and exception-throwing behavior and
- benchmarking support (used to generate results in this paper).

XII. Appendix 2: Performance

Despite a focus on usable interface and direct translation from integer-based fixed-point operations, there is an overwhelming expectation that the source code result in minimal instructions and clock cycles. A few preliminary numbers are presented to give a very early idea of how the API might perform.

Some notes:

- A few test functions were run, ranging from single arithmetic operations to basic geometric functions, performed against integer, floating-point and fixed-point types for comparison.

- Figures were taken from a single CPU, OS and compiler, namely:

```
Debian clang version 3.5.0-10 (tags/RELEASE_350/final) (based on LLVM 3.5.0)
```

```
Target: x86_64-pc-linux-gnu
Thread model: posix
```

- Fixed inputs were provided to each function, meaning that branch prediction rarely fails. Results may also not represent the full range of inputs.
- Details of the test harness used can be found in the source project mentioned in Appendix 1;
- Times are in nanoseconds;
- Code has not yet been optimized for performance.

Types

Where applicable various combinations of integer, floating-point and fixed-point types were tested with the following identifiers:

- `uint8_t`, `int8_t`, `uint16_t`, `int16_t`, `uint32_t`, `int32_t`, `uint64_t` and `int64_t` built-in integer types;
- `float`, `double` and `long double` built-in floating-point types;
- s3:4, u4:4, s7:8, u8:8, s15:16, u16:16, s31:32 and u32:32 format fixed-point types.

Basic Arithmetic

Plus, minus, multiplication and division were tested in isolation using a number of different numeric types with the following results:

name cpu_time
add(float) 1.78011
add(double) 1.73966
add(long double) 3.46011
add(u4_4) 1.87726
add(s3_4) 1.85051
add(u8_8) 1.85417
add(s7_8) 1.82057
add(u16_16) 1.94194
add(s15_16) 1.93463
add(u32_32) 1.94674
add(s31_32) 1.94446
add(int8_t) 2.14857
add(uint8_t) 2.12571
add(int16_t) 1.9936
add(uint16_t) 1.88229
add(int32_t) 1.82126
add(uint32_t) 1.76
add(int64_t) 1.76
add(uint64_t) 1.83223
sub(float) 1.96617
sub(double) 1.98491
sub(long double) 3.55474
sub(u4_4) 1.77006
sub(s3_4) 1.72983
sub(u8_8) 1.72983
sub(s7_8) 1.72983
sub(u16_16) 1.73966
sub(s15_16) 1.85051
sub(u32_32) 1.88229
sub(s31_32) 1.87063
sub(int8_t) 1.76
sub(uint8_t) 1.74994
sub(int16_t) 1.82126
sub(uint16_t) 1.83794
sub(int32_t) 1.89074
sub(uint32_t) 1.85417
sub(int64_t) 1.83703
sub(uint64_t) 2.04914
mul(float) 1.9376

mul(double) 1.93097
mul(long double) 102.446
mul(u4_4) 2.46583
mul(s3_4) 2.09189
mul(u8_8) 2.08
mul(s7_8) 2.18697
mul(u16_16) 2.12571
mul(s15_16) 2.10789
mul(u32_32) 2.10789
mul(s31_32) 2.10789
mul(int8_t) 1.76
mul(uint8_t) 1.78011
mul(int16_t) 1.8432
mul(uint16_t) 1.76914
mul(int32_t) 1.78011
mul(uint32_t) 2.19086
mul(int64_t) 1.7696
mul(uint64_t) 1.79017
div(float) 5.12
div(double) 7.64343
div(long double) 8.304
div(u4_4) 3.82171
div(s3_4) 3.82171
div(u8_8) 3.84
div(s7_8) 3.8
div(u16_16) 9.152
div(s15_16) 11.232
div(u32_32) 30.8434
div(s31_32) 34
div(int8_t) 3.82171
div(uint8_t) 3.82171
div(int16_t) 3.8
div(uint16_t) 3.82171
div(int32_t) 3.82171
div(uint32_t) 3.81806
div(int64_t) 10.2286
div(uint64_t) 8.304

Among the slowest types are `long double`. It is likely that they are emulated in software. The next slowest operations are fixed-point multiply and divide operations - especially with 64-bit types. This is because values need to be promoted temporarily to double-width types. This is a known fixed-point technique which inevitably experiences slowdown where a 128-bit type is required on a 64-bit system.

Here is a section of the disassembly of the s15:16 multiply call:

```
30:   mov    %r14,%rax
      mov    %r15,%rax
      movslq -0x28(%rbp),%rax
      movslq -0x30(%rbp),%rcx
      imul   %rax,%rcx
      shr    $0x10,%rcx
      mov    %ecx,-0x38(%rbp)
      mov    %r12,%rax
4c:   movzbl (%rbx),%eax
      cmp    $0x1,%eax
    â†“ jne    68
54:   mov    0x8(%rbx),%rax
      lea    0x1(%rax),%rcx
      mov    %rcx,0x8(%rbx)
      cmp    0x38(%rbx),%rax
    â†‘ jb     30
```

The two 32-bit numbers are multiplied together and the result shifted down - much as it would if raw `int` values were used. The efficiency of this operation varies with the exponent. An exponent of zero should mean no shift at all.

3-Dimensional Magnitude Squared

A fast `sqrt` implementation has not yet been tested with `fixed_point`. (The naive implementation takes over 300ns.) For this reason, a magnitude-squared function is measured, combining multiply and add operations:

```
template <class FP>
constexpr FP magnitude_squared(const FP & x, const FP & y, const FP & z)
{
    return x * x + y * y + z * z;
}
```

Only real number formats are tested:

float 2.42606
double 2.08
long double 4.5056
s3_4 2.768
s7_8 2.77577
s15_16 2.752
s31_32 4.10331

Again, the size of the type seems to have the largest impact.

Circle Intersection

A similar operation includes a comparison and branch:

```
template <class Real>
bool circle_intersect_generic(Real x1, Real y1, Real r1, Real x2, Real y2, Real r2)
{
    auto x_diff = x2 - x1;
        auto y_diff = y2 - y1;
    auto distance_squared = x_diff * x_diff + y_diff * y_diff;

    auto touch_distance = r1 + r2;
        auto touch_distance_squared = touch_distance * touch_distance;

    return distance_squared <= touch_distance_squared;
}
```

float 3.46011
double 3.48
long double 6.4
s3_4 3.88
s7_8 4.5312
s15_16 3.82171
s31_32 5.92

Again, fixed-point and native performance are comparable.